

	Original	Corrupted Key	Restored @ Key	Restored @ Value	Restored @ Query
Mamba	6.5 74.4 79.4 88.7 90.3	0.6 3.2 5.5 4.2 4.9	1.0 70.9 5.8 3.1 5.3	1.4 3.1 4.6 4.5 5.4	6.5 3.4 79.4 88.7 90.3
Hyena	0.1 0.1 2.6 12.7 29.9	0.1 0.1 0.7 1.5 1.5	0.1 0.1 0.6 1.6 0.9	0.1 0.1 0.4 1.3 1.6	0.1 0.1 2.6 12.7 29.9
H3	0.1 0.2 1.3 1.8 1.0	0.1 0.1 0.5 1.0 0.6	0.1 0.2 0.4 1.8 0.8	0.1 0.1 0.2 1.1 0.7	0.1 0.1 1.3 0.6 1.0
DeltaNet	1.1 66.7 85.3 55.5 84.8	0.1 0.8 1.2 0.6 1.3	1.0 0.8 1.2 0.5 1.1	0.1 1.1 1.6 0.9 0.5	0.1 66.7 85.3 55.5 84.7
Based	21.8 92.0 98.7 98.9 98.5	0.3 2.7 4.1 6.4 7.4	0.1 3.5 5.0 5.8 8.3	21.9 91.9 98.7 98.9 98.5	0.0 2.7 5.7 8.2 7.1
BaseConv	0.0 0.7 3.5 5.8 8.5	0.0 0.5 3.1 6.4 7.8	0.0 0.1 3.3 5.8 8.1	0.0 0.6 3.6 6.0 8.3	0.0 0.5 3.5 6.3 8.2
Attention	1.9 100.0 99.9 100.0 100.0	1.7 5.8 5.5 6.0 4.8	1.7 6.0 4.3 5.3 5.3	2.1 100.0 99.9 100.0 100.0	1.7 5.0 6.4 5.7 4.3
	16 32 64 128 256	16 32 64 128 256	16 32 64 128 256	16 32 64 128 256	16 32 64 128 256
	Model dimension				

Figure 3: **Associative recall**: Likelihood of correct answer without any interventions, after corrupting the key, and after restoring representations at the layer 1 block input with interchange intervention, on AR with vocabulary size 8192 and key–value count of 32. SSMs (except for Based) and Transformers learn different mechanisms.

For each architecture, we intervene at both the input and output each of the following model components f : each layer block, each sequence-mixer (e.g. Attention blocks in each Transformer layer), and each state-mixer (an MLP, except in Mamba, which lacks this component).² We measure to what extent the intervention can restore the likelihood of the correct answer to the query, i.e. we compare **restored likelihood** $p_{f \leftarrow f^*}(y_{\text{true}} \mid \mathbf{c}, \mathbf{o})$ with original likelihood $p(y_{\text{true}} \mid \mathbf{o})$ and corrupted likelihood $p(y_{\text{true}} \mid \mathbf{c})$.

4 UNDERSTANDING AR WITH MECHANISTIC METRICS

We now deploy our mechanistic metrics (§3) on AR. We follow the methodology outlined in §4.1 to create a variety of datasets and train models with various architectures and hyperparameter configurations. See §D for additional experiments not included here.

4.1 METHODOLOGY

Datasets. We generate synthetic pretraining and evaluation datasets for AR. For each setting, the trainset has 100,032 examples and the eval/dev sets have 320 examples. In AR, we use disjoint key and value vocabularies. In each document, we separate the document from the query with a divider token, and provide only a single query. Further details are in §C.

Models. We pretrain small models from scratch. We use the exact architecture implementations from the zoo³ library (Arora et al., 2024b) as well as the DeltaNet implementation from fla⁴ (Yang & Zhang, 2024), except for behaviour-preserving modification of the LM backbone to enable interventions with pyvene⁵ (Wu et al., 2024) on various model-internal components. The LM backbone for all architectures is the same, with pre-norm blocks of alternating sequence mixers and MLPs (except for Mamba, which has no MLP) followed by LayerNorm at the end. We experiment with the following architectures: Attention (Vaswani et al., 2017), BaseConv (Arora et al., 2024a), Based (Arora et al., 2024b), DeltaNet (Yang et al., 2024), H3 (Fu et al., 2023), Hyena (Poli et al., 2023), and Mamba (Gu & Dao, 2024); further details are given in §A.

Training. We minimise next-token prediction cross-entropy loss, and mask the loss on all tokens except the *query*. We use the AdamW optimiser with $\beta = (0.9, 0.999)$, $\epsilon = 10^{-8}$ and no weight decay. We warm up the learning rate for the first 10% of training and then follow a cosine decay schedule to 0 for the remainder of training. We train for 16 epochs with a batch size of 32.

²We are not limited to analysing only these components, however; we do experiment with mixer-internal representations in our analysis.

³<https://github.com/HazyResearch/zoo>

⁴<https://github.com/fla-org/flash-linear-attention>

⁵<https://github.com/stanfordnlp/pyvene>



(a) Original and intervention restoration scores for **Mamba** when varying conv. kernel size; Restoring at the *value* is not shown since none of the models benefited from it. (b) Accuracy on AR for **Based** when varying conv. kernel size or using implicit long conv.

Figure 4: **Ablating short convolution in Mamba and Based:** Accuracy and interchange intervention results when ablating parameters of the short convolution component in Mamba and Based. Conv. size less than 2, using long conv., or no conv., all lead to near-zero performance in both models.

Each experiment trains ≈ 200 models over all hyperparameters. Runtime varies from 0.5 to 5 hours, depending on hardware, task, and architecture. Overall, we used $< 10,000$ GPU-hours in total, on a cluster with various NVIDIA machines (GPU memory 12.3G to 143.8G).

Behavioural metrics. We report behavioural metrics given the model’s predicted probabilities over the vocabulary $\hat{\mathbf{y}} \in \mathbb{R}^{|\Sigma|}$ and the index of the single true answer i . Our main metric is likelihood of the correct answer: $\hat{\mathbf{y}}_i$. We additionally report accuracy in appendices: $\mathbb{1}[\arg \max(\hat{\mathbf{y}}) = i]$.

4.2 (MOST) SSMS DO NOT SOLVE AR WITH INDUCTION

We first run experiments on a standard AR task and show that interchange interventions empirically confirm architectural differences in how AR is solved. We fix the total number of unique keys and values in the vocabulary to be 8192, and present 32 key–value pairs in context. As noted above, our trainset includes 100,032 examples. We vary model dimensionality in $\{16, 32, 64, 128, 256\}$ and sweep LR in the range $[3 \cdot 10^{-5}, 3 \cdot 10^{-2}]$ for each architecture.

Behavioural results. The leftmost panel of Figure 3 demonstrates that answer likelihood on AR cleanly separates Attention, which achieves 100% likelihood and accuracy at $d \geq 32$, from nearly all SSMS. Based solves AR near-perfectly with roughly the same dimension-wise scaling curve as Attention, achieving a maximum likelihood of 98.9% (accuracy of 99.06%). However, Mamba and DeltaNet are close behind and clearly better than other SSMS at AR, albeit achieving a less-than-perfect 90.3% likelihood (91.25% accuracy) at $d = 256$.

Mechanistic analysis. After corrupting the *key*, we restore the original representation at the layer 1 block input at each of various positions (*key*, *value*, and *query*) and observe the resulting likelihood of the true answer. High restored likelihood at the *value* token causally indicates that the model is performing **induction**, whereas *query* indicates **direct retrieval** by the layer 0 block and *key* indicates the layer 1 block, respectively. We report results on the LR-tuned checkpoints of each architecture at each model dimension.

Our results in Figure 3 cleanly separate Attention and Based, which only perform induction, from other SSMS, which either perform direct retrieval (Mamba, DeltaNet, Hyena) or fail to learn the task (H3, BaseConv). SSMS perform direct retrieval at varying layers: the best-performing SSMS almost entirely perform direct retrieval at layer 0 via the *query* token, with only one Mamba checkpoint ($d = 32$) delays retrieval to layer 1. Jelassi et al. (2024) show that direct retrieval in SSMS has asymptotically worse capacity than the induction solution, and this is reflected in performance on AR.

4.3 SHORT CONVOLUTIONS ENABLE AR IN MAMBA AND BASED

We have observed that Attention, Based, and Mamba are the highest-performing architectures on AR. However, their underlying mechanisms differ: Attention and Based learn **induction**, a 2-layer

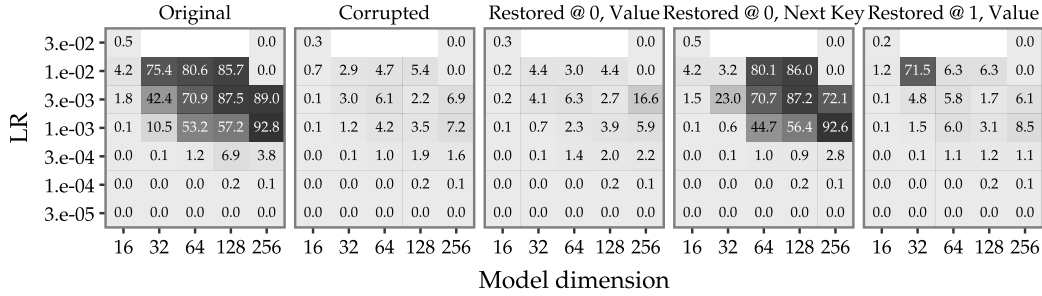


Figure 5: **Interventions on short convolutions in Mamba:** Likelihood of the correct answer for all Mamba checkpoints (varying model dimension and LR) on AR; (from left to right) original performance, after corrupting the *key*, restoring at the layer 0 short conv output at the *value*, same but at the next *key*, and same but at the layer 1 *value* token. (Some runs failed due to high LR.)

mechanism which stores *key*–*value* associations at the *value* token as an intermediate step, whereas Mamba uses **direct retrieval**, a 1-layer mechanism which writes the association to the *query* token.

Importantly, Based and Mamba share a key architectural component: **short convolutions**. Based is a hybrid model with alternating short convolution and linear attention layers, while Mamba applies a short convolution to the layer input before each SSM block. We hypothesise that this architectural component is necessary⁶ for learning AR when using a subquadratic sequence mixer. We conduct experiments on AR where we shorten the convolution kernel size in Mamba (from the default $d_{\text{conv}} = 4$ to $\{3, 2, 1\}$, and deleting it) and replace the Based short convolution with implicitly-parametrised long convolution (Poli et al., 2023).

Results. We report results of our ablations in Figure 4. On Mamba (Figure 4a), we find a step change in task accuracy when increasing d_{conv} from 1 to 2, which introduces previous token information and thus enables AR. Without short convolution, Mamba fails to learn AR. Also, no Mamba checkpoints store the *key* at the *value* (i.e. induction), so direct retrieval persists. Finally, besides $d_{\text{conv}} < 2$ like Mamba, implicit long convolution in Based also significantly harms AR performance (Figure 4b). We thus conclude that short convolutions are necessary for performing AR in Mamba and Based.

4.4 ACTUALLY, MAMBA *does* DO INDUCTION

We have established that (a) Attention and Based perform AR via induction, whereas Mamba and other SSMs use direct retrieval, and (b) Mamba and Based require short convolutions to succeed at AR. However, we have not elucidated what ‘direct retrieval’ is; its only apparent distinction from induction is that it is implemented using a single layer, but what is happening in that layer is unclear.

We have already observed in Based that the association step of AR can be implemented with a short convolution, and then retrieval can be handled by a component with a longer receptive field (linear attention). But §4.2 shows that this results in an induction mechanism in Based: short convolution, just like Attention, performs association by moving information about the key to its associated value. Might short convolution in Mamba serve the same role?

Methodology. We use mechanistic metrics to analyse each layer’s short convolution components in all of our Mamba checkpoints (sweeping model dimension and LR). To enable Mamba-internal interventions, we use a native PyTorch implementation of Mamba and load in the weights from our hardware-optimised training runs; we confirm that unintervened performance is unchanged. As in §4.2, we corrupt the *key*, and attempt to restore *key* information by intervening at the hidden states outputted by the short convolution component in each layer. We report the likelihood of the correct answer after intervention, averaged over 64 inputs.

Results. Our results in Figure 5 show that short convolution does move the *key* information; however, only a single checkpoint ($d = 32$, $\text{LR} = 10^{-2}$) actually moves *key* information to the *value* (as expected in induction, cf. Attention), and that too in layer 1; instead, we observe in the remaining 9

⁶Since Hyena also has a short convolution, this may not be *sufficient* for good performance on association.